

Lab. 4
Connecting Node.js to MySQL

Content

1. Recap: from SQL by hand to SQL from a program
2. Setting up a Node.js project; installing mysql2
3. Connecting, and async / await
4. Running SELECT and reading rows as JavaScript objects
5. INSERT / UPDATE / DELETE from code
6. Parameterized queries
7. A complete script: connect, query, handle errors, close
8. Where we're going next (Lab 5)
9. Summary
10. Project discussion (group formation)

Recap: From SQL by Hand to SQL from Code

- In Lab 1 you wrote JavaScript & in Lab 3 you wrote SQL by hand in MySQL Workbench.
- Today we join them, a Node.js program sends those same SQL statements to MySQL.
- The driver we use is mysql2, a small library that connects to the server and returns rows.
- Same database, same SQL, now run from code instead of typed into Workbench.

Setting Up a Node.js Project; Installing mysql2

Why Talk to the Database from a Program?

- Real applications don't type SQL by hand, a program does it for them.
- Queries become dynamic: the value in a WHERE comes from a user, a form, or a file.
- The rows come back as JavaScript objects you can loop over, filter and display, just like Lab 1's arrays.
- Workbench is for designing and exploring; your program is for doing it automatically.

Setting Up a Node.js Project; Installing mysql2 (Contd.)

Project Setup

Make a folder, start a project, and install the mysql2 driver. Run these in the VS Code terminal.

```
mkdir lab4-db && cd lab4-db
npm init -y
npm install mysql2
```

npm install adds mysql2 to node_modules (project's dependencies live here) and lists it in package.json. Don't commit node_modules (just increases the repo size in github)

Setting Up a Node.js Project; Installing mysql2 (Contd.)

Connecting with mysql2

Open a connection with your server details, then use it to run queries. We use the promise API.

```
// app.js
const mysql = require('mysql2/promise');

async function main() {
  const conn = await mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: 'your_password',
    database: ‘lab3'   // your Lab 3 schema
  });
}
main();
```

host/ user/ password match your MySQL server; database is the schema you built in Lab 3 (or use the given zip file).

Connecting, and async / await

- Talking to a database takes time, so mysql2 returns a promise, a value that arrives later.
- await pauses until that value is ready, then hands you the result — no callbacks to nest.
- A function that uses await must be marked async; we put our code inside async function main() { ... }.
- Rule of thumb: every query line starts with await.

Running SELECT and Reading Rows as JavaScript Objects

Send a SELECT and read the rows it returns. query() gives back an array of rows.

```
const [rows] = await conn.query('SELECT * FROM students');

console.log(rows.length, 'students');

for (const s of rows) {
  console.log(s.name, '-', s.major);
}
```

query() returns [rows, fields]; we de-structure the first part. Each row is a plain object.
**** Don't forget to put it inside the async main function (just after connection object).*

Running SELECT and Reading Rows as JavaScript Objects (Contd.)

Rows Are Just Arrays of Objects

- rows is an array; each element is an object with one property per column.
- rows[0].name, rows[0].major, exactly the objects and arrays you met in Lab 1.
- So everything from Lab 1 still works: for...of, .map(), .filter(), .length.
- The array you printed by hand in Lab 1 is now filled straight from the database.

Running SELECT and Reading Rows as JavaScript Objects (Contd.)

Lab Exercise 1: *Connect and read*

- Create lab4-db, run npm init -y and npm install mysql2.
- Write ex1.js that connects to your Lab 2/3 schema.
- Run SELECT * FROM students and print each student's name and major.
- Run it with node ex1.js and check the rows print in the terminal.

Starter code:

```
const mysql = require('mysql2/promise');

async function main() {
  const conn = await mysql.createConnection({ /* ... */ });
  const [rows] = await conn.query('SELECT * FROM students');
}
main();
```

Parameterized queries

When a value comes from outside, never glue it into the string. Use ? and pass the values separately.

```
const major = 'Data Science';

const [rows] = await conn.execute(
  'SELECT name FROM students WHERE major = ?',
  [major]
);
```

execute() fills each ? from the array, in order, quoting safely. One ? per value.

Parameterized queries (Contd.)

Why the ? Matters

- Building SQL by joining strings is unsafe and error-prone where quotes and apostrophes break it.
- Worse, a hostile value can change what your query does and this is SQL injection.
- The driver handles quoting and escaping for you when you pass values through ?.
- Habit for this course: every value from a variable (input from user) goes through a ? placeholder.

INSERT, UPDATE and DELETE from Code

Write operations use ? the same way; the result tells you what happened.

```
const [r1] = await conn.execute(
  'INSERT INTO students (name, major, year) VALUES (?, ?, ?)',
  ['Mali', 'Data Science', 1]);
console.log('new id:', r1.insertId);

const [r2] = await conn.execute(
  'UPDATE students SET year = year + 1 WHERE id = ?', [r1.insertId]);
console.log('rows changed:', r2.affectedRows);
```

insertId is the AUTO_INCREMENT id of the new row; affectedRows counts the rows touched.

INSERT, UPDATE and DELETE from Code (Contd.)

Lab Exercise 2: *Write with placeholders*

- Insert a new student with execute() and ? placeholders; print the insertId.
- Run a SELECT with a ? to list only the students in a major you store in a variable.
- Update that new student's year by one and print affectedRows.
- Confirm in Workbench that the row really changed.

Starter code:

```
await conn.execute(
  'INSERT INTO students (name, major, year) VALUES (?, ?, ?)',
  ['____', '____', __]);
```

A Complete Script: connect, query, Handle errors, Close

Putting It Together: A complete script: connect, do some work, then always close the connection.

```
const mysql = require('mysql2/promise');

async function main() {
  const conn = await mysql.createConnection({ /* config */ });
  const [rows] = await conn.query('SELECT * FROM courses');
  for (const c of rows) console.log(c.title, c.credits);
  await conn.end();
}
main();
```

One async main() holds the whole flow; conn.end() closes the connection when you're done.

A Complete Script: connect, query, Handle errors, Close (Contd.)

Error Handling and Closing: Wrap the work in try / catch / finally so the connection always closes, even on an error.

```
async function main() {
  const conn = await mysql.createConnection({ /* config */ });
  try {
    const [rows] = await conn.query('SELECT * FROM students');
    console.log(rows);
  } catch (err) {
    console.error('Query failed:', err.message);
  } finally {
    await conn.end();
  }
}
```

finally runs whether or not there was an error, the right place to close the connection.

A Complete Script: connect, query, Handle errors, Close (Contd.)

Lab Exercise 3: *A complete script*

- Write one async main() that connects, runs at least two queries, and closes the connection.
- Wrap the queries in try / catch / finally; close the connection in finally.
- Make one query deliberately fail (a wrong table name) and read the caught message.
- Fix it, re-run, and confirm the script ends cleanly.

Starter code:

```
try {// your queries here
} catch (err) { console.error(err.message);
} finally {
  await conn.end();
}
```

A Complete Script: connect, query, Handle errors, Close (Contd.)

Good Habits

- Parameterize every value from a variable with ? , never build SQL by string concatenation.
- Keep your connection config in one place; don't hard-code passwords across many files.
- Always close the connection (conn.end()), ideally in a finally block.
- Check insertId and affectedRows so your program knows the write actually happened.

Where we're going next (Lab 5)

- You can now connect, read and write to MySQL from a Node.js program, the heart of any data-driven app.
- In Lab 5 we wrap these queries into reusable functions and expose them over the web with Express.
- The same SQL and the same mysql2 calls become the engine behind a small web service.

Summary

- npm install mysql2; connect with mysql2/promise and createConnection({ ... }).
- await every query; rows come back as an array of objects just like Lab 1.
- Pass every variable value through a ? placeholder with execute(), safe and correct.
- insertId / affectedRows report what a write did; close with conn.end() in finally.
- Complete Exercises 1–3 in VS Code during class, against your Lab 3 database.

Project Discussion

Project evaluating criteria and requirements

1. 3 - 4 students per group.
2. Entity - Relationship diagram (at least 5 entities & 3 relationship sets: show in Proposal & relation explained in demonstration).
3. HTML / CSS / Node js or Any (front end: mockup in proposal)
4. Use only MySQL server for the database server (must have at least 1 stored procedure, 1 trigger: on final demonstration)
5. Database security implemented (hashing sensitive information: explained on final demonstration)
6. At least 2 app pages (At proposal and demonstration)
7. With all operations - Create(Insert) / Read (Select) / Update / Delete (At proposal & Demonstration explained: at least one of each)
8. Developer, Administrator and a user should have different access privileges (Access Control) (At demonstration)
9. Make slides in the proposal and the final will be the demonstration of the finished project

Thank you & Assignment time…